

Brain Capture

iOS Shortcut reliability fix — step-by-step walkthrough

What this fixes: the current Shortcut shows a hardcoded `Hello, world!` notification on completion regardless of whether the GitHub API call actually succeeded. Failed captures look identical to successful ones, so you only notice notes are missing when you go looking. This guide replaces the silent fail with a branching success/failure handler and an Apple Notes fallback, so no dictation can ever vanish unnoticed.

Time required: 10–15 minutes. **Where:** iOS Shortcuts app, edit the *Brain capture* Shortcut. **Result:** every capture self-reports success or failure; failures buffer to a local note named *Brain Failures*.

Before you start

This guide assumes the existing Shortcut works most of the time and the failure mode is the silent-on-error notification. If the Shortcut never works, the bug is somewhere earlier (token, URL, or PUT body) and this guide will not help. Run the Shortcut once; check whether anything new appears in `~/Brain/Inbox/` on your laptop. If nothing arrives even on the happy path, fix that first.

1. The new flow at a glance

After the edit, the Shortcut keeps everything from *Dictate text* through *Get contents of URL* exactly as it is. Only the tail changes — instead of one hardcoded notification, you add a branching check on the GitHub API response.

Existing actions (no change)

```
Dictate text
Encode dictated text with base64
Current Date
Format Current Date
Text -> https://api.github.com/repos/HugoIdle/Brain/contents/Inbox/[Date].md
Get contents of URL (PUT, with token + JSON body)
```

New tail (replaces the Hello, world! notification)

```
Get Dictionary Value key: sha from: Contents of URL
If [Dictionary Value] has any value
    Show Notification "Brain capture saved"
Otherwise
    Get Dictionary Value key: message from: Contents of URL
    Append to Note note: "Brain Failures"
    body: date + dictated text + error
    Show Notification "FAILED. Saved to Notes."
End If
```

The principle: GitHub's Contents API returns a JSON body with a sha field on success (HTTP 201) and an error/message field on failure (any non-2xx). Branching on whether sha exists is the cleanest way to detect outcome from inside Shortcuts.

2. The edit, step by step

Open the iOS Shortcuts app, tap the existing **Brain capture** Shortcut, then tap the edit icon (the three sliders or the pencil, depending on iOS version) to enter edit mode.

STEP 1 Delete the old notification

Scroll to the bottom. Find the **Show Notification** action with text `Hello, world!`. Long-press it and choose **Delete**, or swipe it left and tap the bin icon. This is the only thing being removed.

STEP 2 Add: Get Dictionary Value (extract the sha)

Tap the search bar at the bottom and search **Get Dictionary Value**. Add it.

- Set **Get** to **Value**
- Set **Key** to `sha`
- Set **Dictionary** to the *Contents of URL* output from the previous action (tap the variable bubble and pick the URL response)

If GitHub returned 201 Created, the response JSON contains a `content.sha` field. The Get Dictionary Value action extracts it (or returns nothing on failure, which is what the next step keys off).

STEP 3 Add: If — branch on sha presence

Search and add **If**. Configure:

- Input: the **Dictionary Value** output from Step 2
- Condition: **has any value**

iOS will automatically add *Otherwise* and *End If* markers. Anything you place between *If* and *Otherwise* runs only on success.

STEP 4 Inside If: success notification

Between *If* and *Otherwise*, search and add **Show Notification**.

- Title (optional): Brain capture
- Body: Saved to Inbox
- Optional: enable **Play sound** for an audible confirmation

Optional polish: prefix with a tick character so the notification reads "OK -- Saved to Inbox". Use whichever character your lock-screen renders well; the iOS notification banner uses your system font.

STEP 5 Inside Otherwise: extract the error message

Between *Otherwise* and *End If*, add **Get Dictionary Value** again.

- Get: **Value**
- Key: message
- Dictionary: the same *Contents of URL* output (tap variable bubble)

GitHub's error responses include a top-level message field with text like *Bad credentials*, *API rate limit exceeded*, or *Validation Failed*. This step extracts that string so the failure notification and the local note can include it.

STEP 6 Inside Otherwise: append the dictation to a fallback note

Search and add **Append to Note** (this is the Apple Notes integration action).

- Note: tap and choose **Ask Each Time** first, run the Shortcut once, then come back and lock it to a note named Brain Failures. If that note doesn't exist yet, create it manually in Apple Notes first.
- Body: combine three things using the variable picker — Formatted Date, then a newline, then Dictated Text, then a newline, then Error: [Dictionary Value from Step 5]

After this step, every failed capture is captured locally on the phone in plain text. Even if the phone is offline, the note is in Apple Notes and will sync to iCloud when you regain connectivity. The dictation is unrecoverable from GitHub at this point but fully recoverable from this note.

STEP 7 Inside Otherwise: failure notification

Add another **Show Notification** action below Append to Note.

- Title: Brain capture FAILED
- Body: combine Saved to Notes -- + the Dictionary Value from Step 5 (the error message)
- Enable **Play sound** so failures are audibly distinct from successes

You should now have a complete If/Otherwise/End If block with success on one branch and three actions on the failure branch.

3. Test it before relying on it

Three quick tests to confirm both branches work. Do not skip these — the whole point of the rebuild is observability, and the only way to verify observability is to trigger a known failure.

Test 1 — Happy path

- Run the Shortcut, dictate a short test phrase like *'shortcut test 1'*
- Expect: notification reads **Saved to Inbox**
- On the laptop, run `git -C ~/Brain pull` and check `~/Brain/Inbox/` — a new `.md` file should be present with your dictation

Test 2 — Forced failure (token)

- Edit the Shortcut. Open *Get contents of URL*. Temporarily change one character of the GitHub PAT (in the Authorization header) to break it
- Run the Shortcut, dictate *'shortcut test 2 should fail'*
- Expect: notification reads **FAILED -- Bad credentials** (or similar). The Apple Notes app should now have a *Brain Failures* note with the dictation and error
- **Restore the PAT character** before continuing

Test 3 — Forced failure (offline)

- Enable Airplane Mode
- Run the Shortcut, dictate *'shortcut test 3 offline'*
- Expect: failure notification, dictation buffered to *Brain Failures* note
- Disable Airplane Mode. The note remains in Apple Notes — capture is recoverable

Why these three?

Test 1 confirms success still works. Test 2 confirms the API-error branch (the most likely real-world failure: token expired, rate limited, scope changed). Test 3 confirms the offline branch (the silent-loss case you actually mentioned). If all three behave as expected, the Shortcut is reliable.

4. Optional polish: retry loop

Skip this section unless you find that real-world failures are mostly transient (single network blip, brief connectivity drop). For permanent failures (token expired, scope removed) retries do nothing. The fallback note is sufficient on its own.

If you want retries: wrap *Get contents of URL* in a **Repeat 3 Times** block. Inside the block, after the URL call, add an **If** with condition *Contents of URL contains 'sha'* and put a **Stop Shortcut** action inside the If branch. Add a **Wait 2 seconds** action just before the URL call, so retry attempts space out.

```
Repeat 3 Times
  Wait 2 seconds (skip on iteration 1 – see note below)
  Get contents of URL (existing PUT call)
  If [Contents of URL] contains "sha"
    Stop Shortcut and Output [Contents of URL]
  End If
End Repeat
(then continue with the success/failure branch from Section 2)
```

Note on the wait: iOS Shortcuts has no clean way to skip the first iteration's wait. The simplest workaround is to set the wait to 1 second and accept that successful captures take a second longer. Or: precompute a Counter and use an *If [counter] is greater than 1* wrapper around the wait. Most users will not bother.

5. Maintenance

Auditing the failure note

Open the *Brain Failures* note in Apple Notes once a week or whenever you suspect a capture went missing. If the note is empty, your capture pipeline is healthy. If it has entries, copy the dictations from the note into `~/Brain/Inbox/` manually on the laptop, then clear the note. The dictation never enters the GitHub-side audit trail (because the API call failed), but it is preserved in plain text on the phone.

When the PAT expires

GitHub fine-grained PATs have a finite expiry. When yours expires, the Shortcut will start populating the *Brain Failures* note with `Bad credentials` errors. That is the system telling you it is time to regenerate the PAT and update the Authorization header in *Get contents of URL*. Set a calendar reminder for one week before the PAT expires, or accept that the failure note will warn you.

Filename collisions

If your *Format Current Date* string is short enough that two captures within the same time-window produce the same filename (e.g. you set it to `yyyy-MM-dd` rather than `yyyy-MM-dd 'T' HH-mm-ss`), the second capture's PUT will fail with `422 Validation Failed` because GitHub requires a SHA to update an existing file. With the new failure-note flow, you will see this error and can fix the format string. Today's inbox file `2026-05-04T16-15-39.md` suggests your current format includes seconds, which is collision-safe in practice.

Generated by Claude Code, document-skills:pdf. Source: live inspection of the Brain capture Shortcut screenshot, 2026-05-04.